Final Report – Machine Learning Project

Combining Evolutionary Learning, Active Learning and Ensemble Learning to solve the Higgs Bozon Detection

Hugo BONNEL
Mathieu COWAN

ESILV – A5

Academic Year 2025–2026

Contents

1 Git repository

The source code of our project is available on the following url address : `https://github.com/matcoc0/MachineLearningProject`. The following elements will be found on this git repository :

- The README.md explaining the main information on the source code, how to launch it, etc.
- The main.py, that you need to run to launch our project.
- The documentation explaining in detail our choices of implementations on a technical standpoint
- The `/src` folder containing all of the other python files such as pipeline.py (implementation of the retained models) or ga.py where GA and GA+AL methods were implemented, and so on - detailled in documentation.
- The `/reports` folder with the `/data` subfolder containing of our EDA results (csv, json & png files representing information such as correlation, missing values...), the `/results` subfolder containing json, csv and png files representing the different metrics for comparison of performance (retained model and experimental models).
- The `/training` folder with the csv, json and the png files representing the evolution of the F1 metric during training, allowing to have a better comprehension of convergence, divergence and stabilisation of our principal metric in this product

2 Introduction

The discovery of the Higgs boson represents a major milestone in modern particle physics. Its identification in high-energy collision data relies on the analysis of large-scale, high-dimensional and noisy datasets. The Higgs Boson Machine Learning Challenge dataset provides a realistic benchmark for studying advanced classification methods aimed at distinguishing signal events from background noise.

Traditional machine learning approaches often face limitations in this context due to non-linear decision boundaries, feature correlations, class imbalance, and high labeling costs. Evolutionary learning techniques, such as Genetic Programming (GP), offer a flexible alternative by evolving symbolic decision functions capable of modeling complex patterns. However, GP methods are computationally expensive and sensitive to the amount of labeled data.

To address these limitations, this project integrates two complementary paradigms: Active Learning (AL) and Ensemble Learning (EL). Active Learning improves efficiency by iteratively selecting the most informative samples from an unlabeled pool, while Ensemble Learning enhances robustness by combining multiple GP models through voting strategies.

The goal of this report is to implement and compare three approaches: (i) a baseline Genetic Programming classifier, (ii) Genetic Programming combined with Active Learning, and (iii) an ensemble of GP models trained with Active Learning. The comparison focuses on predictive performance and computational cost.

3 Dataset Description

The experiments conducted in this project are based on the Higgs Boson Machine Learning Challenge dataset, originally released to support the development of advanced classification methods for high-energy physics applications. The dataset simulates proton–proton collision events recorded at the Large Hadron Collider (LHC), with the goal of distinguishing Higgs boson signal events from background processes.

Each observation corresponds to a single collision event described by a set of numerical features derived from low-level detector measurements and high-level reconstructed variables. After removing non-feature columns, the dataset contains **30 numerical features** and one target label. The target variable `Label` is binary, where:

- `b` represents background events,
- `s` represents signal events corresponding to Higgs boson decays.

The dataset is characterized by a strong class imbalance. As illustrated in Figure 1, background events significantly outnumber signal events. This imbalance reflects realistic experimental conditions and motivates the use of evaluation metrics beyond simple accuracy, such as F1-score, recall, precision and accuracy.

Several features contain missing values, encoded in the original dataset using the value -999. The proportion of missing values varies across features, as summarized in the exploratory analysis. This characteristic requires dedicated preprocessing steps prior to model training.

For all experiments, the dataset is split into training and test sets using a stratified strategy, preserving the original class distribution. The test set represents 20% of the data and is held out throughout training and model selection to provide an unbiased evaluation of generalization performance.

A detailed exploratory analysis of feature distributions, correlations, missing values, and train–test distribution shifts is presented in the following section.

4 Exploratory Data Analysis (EDA)

This section presents an exploratory analysis of the dataset used in this project. All analyses and visualizations were automatically generated by the EDA pipeline and saved in the `reports/data/` directory.

4.1 Dataset Overview

A global overview of the dataset is provided in the file `dataset_overview.json`. This file summarizes the main structural properties of the dataset, including:

- the total number of samples,
- the number of features,
- the list of feature names,
- the data types associated with each feature.

This confirms that the dataset is composed of numerical features only, with a single categorical target variable (`Label`), and contains **818,238 samples** and **30 input features**.

4.2 Label Distribution

The class distribution is analyzed using the files:

- `label_counts.csv`,
- `label_distribution.png`.

Figure 1 shows the number of samples per class. The dataset is moderately imbalanced, with a majority class (`b`) and a minority class (`s`).

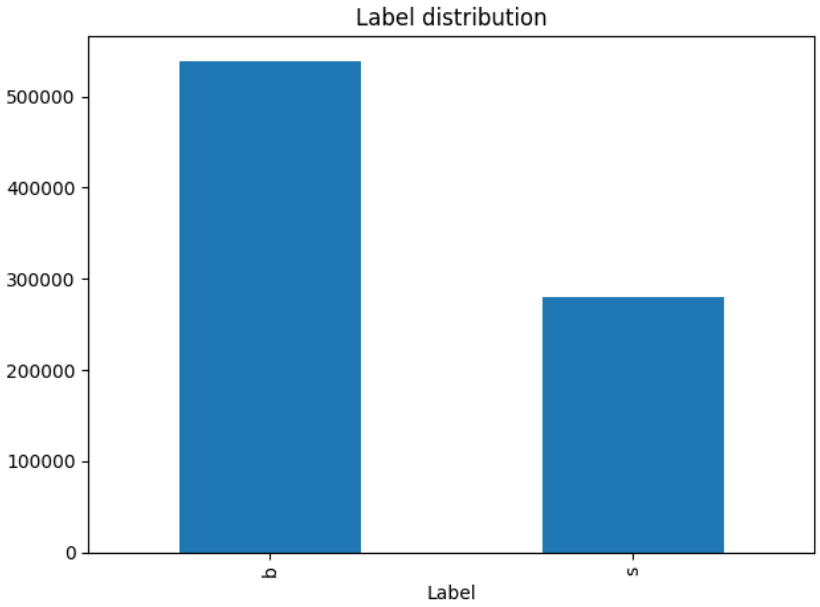

Figure 1: Distribution of the labels throughout the Higgs Boson dataset

This class imbalance motivates the use of the F1-score as the main evaluation metric and justifies the exploration of Active Learning strategies.

4.3 Missing Values Analysis

The proportion of missing values per feature is reported in `missing_values_ratio.csv`.

The analysis shows that some features contain missing values, while others are fully observed. Missing values are not uniformly distributed across features, which motivates the need for a robust preprocessing strategy before training.

4.4 Feature Summary Statistics

The file `feature_summary.csv` provides descriptive statistics for each feature, including:

- minimum and maximum values,
- mean and standard deviation,
- proportion of missing values,
- detection of potential outliers.

This analysis highlights large differences in feature scales, which may impact model training and justifies the use of scale-invariant models such as Genetic Programming.

4.5 Correlation Analysis

Feature correlations are analyzed using:

- `correlation_matrix.csv`,
- `correlation_heatmap.png`.

Figure 2 shows the correlation heatmap between all numerical features.

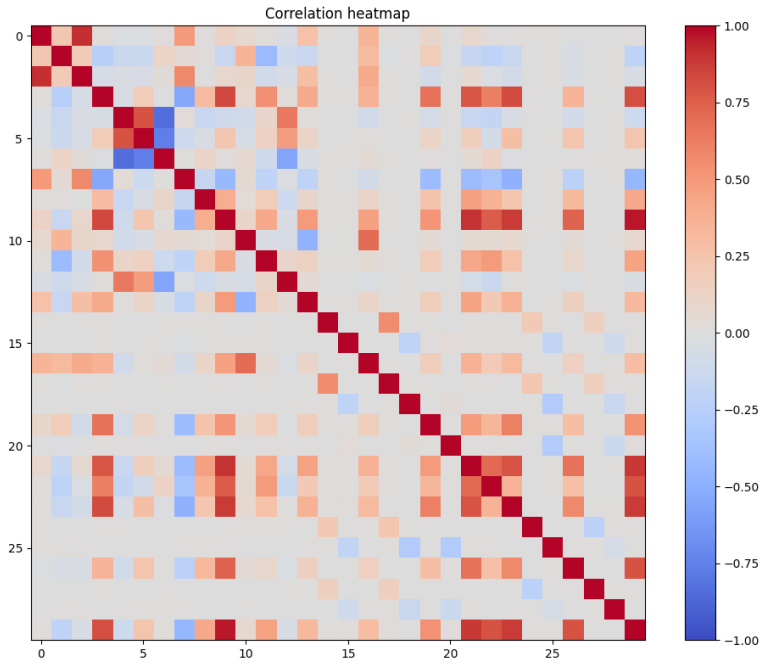

Figure 2: Correlation Heatmap of the Higgs Boson features

Several groups of highly correlated features can be observed, indicating a certain level of redundancy in the feature space. However, no extreme correlations suggesting trivial feature elimination are observed.

4.6 Principal Component Analysis (PCA)

A Principal Component Analysis was performed to better understand the structure of the feature space. The results are saved in:

- `pca_2d_projection.png`,
- `pca_explained_variance.png`.

Figure 3 shows the projection of the data onto the first two principal components, colored by class label.

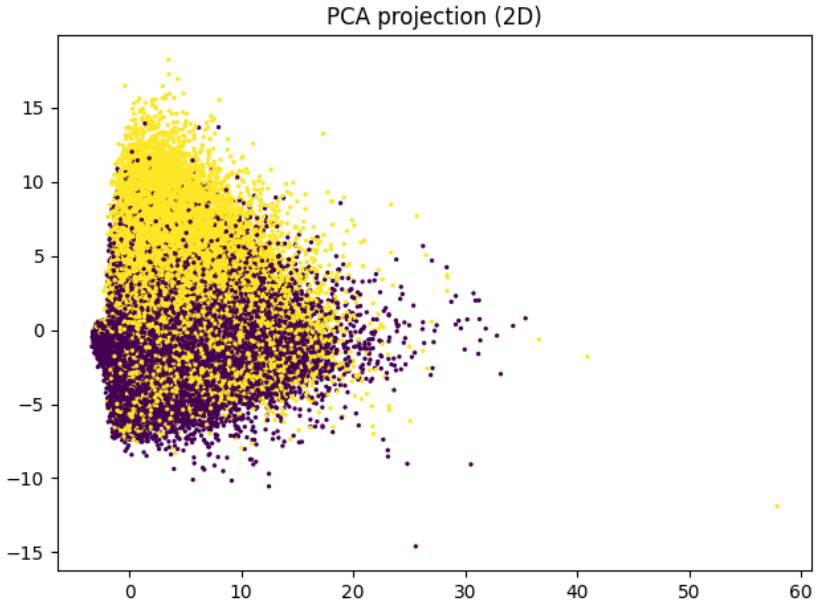

Figure 3: PCA 2D projection

The classes are not linearly separable in this reduced space, indicating that non-linear decision boundaries are required.

Figure 4 presents the explained variance ratio of the first principal components.

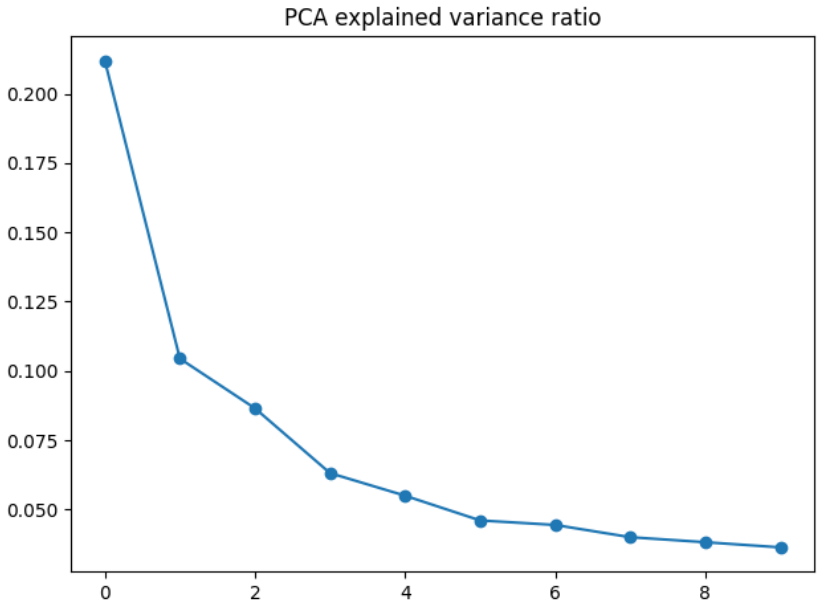

Figure 4: PCA explained variance

The variance is distributed across many components, confirming that the dataset has a relatively high intrinsic dimensionality.

4.7 Train/Test Distribution Shift

The file `data/train_test_shift.csv` compares feature statistics between the training and test splits.

This analysis shows no significant distribution shift between the two sets, indicating that the train/test split is representative and suitable for model evaluation.

5 Data Preprocessing

This section describes the preprocessing pipeline applied to the dataset before model training. The preprocessing steps were designed to ensure numerical stability, reproducibility, and fair evaluation while remaining simple and transparent.

All preprocessing operations are implemented in the file `data.py` and are applied identically for all models.

5.1 Feature Selection

Several non-informative or metadata-related columns are removed from the dataset prior to training. These include identifiers and competition-specific fields that do not carry predictive information.

The removed columns are:

- `EventId`,
- `Weight`,
- `KaggleSet`,
- `KaggleWeight`.

After this step, the dataset contains **30 numerical input features**.

5.2 Label Encoding

The target variable `Label` is encoded into a binary numerical format:

- `b` $\rightarrow 0$,
- `s` $\rightarrow 1$.

This encoding allows direct compatibility with binary classification metrics such as precision, recall, and F1-score.

5.3 Missing Value Handling

Missing values in the dataset are originally encoded using the sentinel value `-999`. These values are first replaced by `NaN`.

A mean imputation strategy is then applied using statistics computed **only on the training set**. This prevents information leakage from the test set into the training process.

The same imputation parameters are subsequently applied to the test set.

5.4 Feature Scaling

After imputation, all features are standardized using a `StandardScaler`. Each feature is transformed to have zero mean and unit variance.

As for imputation, the scaler is fitted exclusively on the training data and then applied to the test data using the same parameters.

Although Genetic Programming models are generally less sensitive to feature scaling than linear models, this step improves numerical stability and ensures consistent behavior across different operators.

5.5 Train/Test Split

The dataset is split into training and test sets using a stratified split, in order to preserve the original class distribution in both subsets.

The split configuration is as follows:

- test size: 20%,
- stratification on the target label,
- fixed random seed for reproducibility.

This results in:

- 654,590 samples in the training set,
- 163,648 samples in the test set.

5.6 Summary

The preprocessing pipeline is deliberately kept simple and robust. It ensures:

- no data leakage between training and test sets,
- consistent handling of missing values,
- numerical stability for evolutionary operators,
- full reproductibility of experimental results.

No aggressive feature engineering or dimensionality reduction is applied at this stage, as the goal is to evaluate the learning capacity of Genetic Programming and Active Learning methods on the original feature space.

6 Methodology

This section describes the complete machine learning pipeline implemented for the Higgs boson classification task, from data preprocessing to model training, ensemble learning, and evaluation. The methodology strictly follows the structure of the implemented codebase and ensures full reproducibility.

6.1 Overall Pipeline Structure

The global execution flow is orchestrated by the main script (`main.py`), which sequentially performs:

- Exploratory Data Analysis (EDA)
- Data preprocessing and splitting
- Genetic Algorithm (GA) training
- Genetic Algorithm with Active Learning (GA+AL)
- Ensemble Learning based on GA+AL
- Optional experimental variants

The core logic is implemented in `pipeline.py`, ensuring a clear separation between production models and experimental extensions.

6.2 Data Preprocessing

Data preprocessing is handled in the module `data.py`. The original dataset is loaded from `atlas-higgs-challenge-2014-v2.csv.gz`

The following preprocessing steps are applied:

1. **Removal of non-feature columns**: identifiers and competition-related metadata are removed.
2. **Label encoding**: the original labels (`b`, `s`) are mapped to binary values (0, 1).
3. **Missing value handling**: missing values encoded as `-999` are replaced by `NaN`, followed by mean imputation.
4. **Feature scaling**: numerical features are standardized using z-score normalization.
5. **Train/test split**: the dataset is split into training and testing sets using a stratified split to preserve class proportions.

All preprocessing parameters are learned exclusively from the training set to avoid data leakage.

6.3 Genetic Programming Model

The primary classifier is based on Genetic Programming, implemented using the DEAP framework in `ga.py`.

6.3.1 Individual Representation

Each individual represents a symbolic mathematical expression composed of:

- Arithmetic operators: addition, subtraction, multiplication
- Protected division operator
- Unary negation
- Ephemeral random constants

The trees operate directly on the input feature space, producing continuous outputs that are thresholded to obtain binary predictions.

6.3.2 Fitness Function

The fitness of each individual is evaluated using the F1-score on the current training set:

$$\text{fitness} = \text{F1-score}(y, \hat{y})$$

This choice ensures robustness to class imbalance, which is observed in the dataset.

6.3.3 Evolutionary Process

The evolutionary loop follows a standard genetic algorithm structure:

- Tournament selection
- One-point crossover
- Uniform mutation
- Static depth constraint to prevent tree bloat

At each generation, statistics are logged, including:

- Generation index
- Best F1-score
- Mean population F1-score
- Current training set size

These logs are saved to `[reports/training/ga_log.csv]`

While Genetic Programming provides a flexible and expressive modeling framework, its reliance on a fully labeled training set and its computational cost motivate the integration of Active Learning, introduced in the next section.

6.4 Active Learning Strategy

An Active Learning extension is implemented within the same genetic framework.

6.4.1 Initial Labeled Set

At the beginning of training, only a fraction α of the training data is labeled:

$$\alpha = \texttt{al\_initial\_fraction}, \qquad |\mathcal{L}_0| = \lfloor \alpha \, |\mathcal{D}_{train}| \rfloor$$

The remaining samples form an unlabeled pool:

$$|\mathcal{U}_0| = |\mathcal{D}_{train}| - |\mathcal{L}_0|$$

6.4.2 Query Strategy

Every `[al_interval]` generations, new samples are selected from the pool according to an uncertainty-based strategy:

- Samples with predictions closest to zero are considered the most uncertain
- These samples are added to the labeled training set

This process dynamically increases the training set size, as reflected in the training logs.

6.5 Ensemble Learning

To improve robustness and generalization, an ensemble model is constructed from the GA+AL population.

6.5.1 Ensemble Construction

The ensemble selects the top-performing individuals from the final population and combines their outputs using soft voting:

$$\hat{y} = \mathbb{1}\left(\frac{1}{K}\sum_{i=1}^{K} f_i(x) > 0\right)$$

where K is the ensemble size.

6.5.2 Production Choice

Soft voting is retained for the final "production" model due to its stability - however here the weighted and the hard voting amount to the same results.

6.6 Evaluation Protocol

All models are evaluated on a held-out test set that is never used during training or active learning.

The following metrics are reported:

- Accuracy
- Precision
- Recall
- F1-score

Results are saved in `[reports/results/metrics.csv]`

6.7 Experimental Variants

Additional experiments are optionally executed when enabled in the configuration:

- Alternative GA hyperparameters
- Alternative Active Learning strategies
- Alternative ensemble voting schemes

These experiments are isolated from the production pipeline and stored in `[reports/results/experime`

6.8 Model Selection Strategy and Experimental Workflow

The overall objective of this project is not only to achieve strong predictive performance, but also to design a robust, reproducible, and extensible machine learning pipeline. To this end, a clear separation is enforced between **production (baseline) models** and **experimental variants**.

A baseline–experiment workflow is adopted for the following reasons:

- ensure reproductibility and stability of the retained models,
- preserve a clean production pipeline while allowing research-oriented exploration.

Production models.

The production pipeline includes three retained approaches:

- a Genetic Algorithm (GA) baseline,
- a GA with Active Learning (GA+AL),
- an ensemble model based on GA+AL using soft voting (GA+AL+EL).

These models are executed deterministically within the main pipeline and serve as stable reference points. Their results are saved separately and are used as benchmarks for all subsequent analyses.

Experimental variants.

In parallel, a dedicated experimental workflow is implemented to explore alternative configurations without impacting the production setup. These experiments include:

- structural constraints on GP trees (e.g., reduced maximum height),
- increased population sizes,
- modified Active Learning acquisition strategies,
- alternative ensemble voting mechanisms (hard, weighted, diversity-based).

All experimental runs are executed only after the production models have completed, using the same preprocessed dataset. Results are stored in a separate directory and explicitly labeled as *experimental* to avoid confusion with production metrics.

Table 1: Comparison of approaches on classification performance

Approach	Accuracy	Precision	Recall	F1	Training time
GA	0.702	0.544	0.789	0.644	436.020
GA+AL	0.695	0.538	0.757	0.629	59.770
GA+AL+EL (soft)	0.695	0.538	0.757	0.629	59.770

Rationale.

At this stage, the goal is not to immediately replace production models with experimental variants, but rather to:

- assess the sensitivity of the Genetic Programming framework to architectural and sampling choices,
- evaluate performance–cost trade-offs,
- identify promising directions for future optimization.

This structured approach allows the project to remain extensible and scientifically grounded, while keeping the production pipeline simple, interpretable, and reproducible.

7 Results and Analysis

7.1 Production Model Results

This section presents the results obtained by the retained production models. These approaches constitute the stable reference baseline of the project and are executed deterministically within the main pipeline.

Three production models are evaluated:

- a Genetic Algorithm baseline (GA),
- a Genetic Algorithm with Active Learning (GA+AL),
- an ensemble model based on GA+AL using soft voting (GA+AL+EL).

All models are evaluated on the same held-out test set using accuracy, precision, recall, and F1-score.

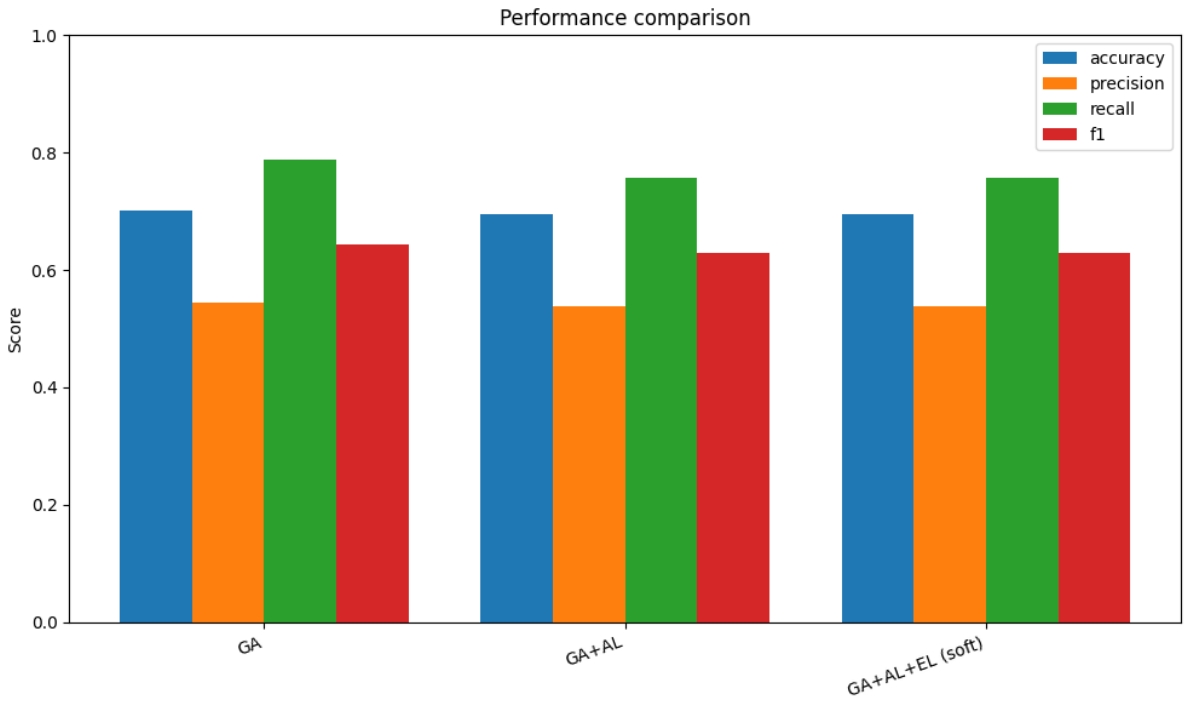

Figure 5: Performance comparison across experimental variants of GA, GA+AL and GA+AL+EL.

7.2 General Evaluation of performances – Highest Accuracy and F1-score and variation of precision/recall between approaches

Overall, the baseline Genetic Algorithm (GA) achieves the highest performance among the evaluated approaches, with the best accuracy (approximately 0.70) and the highest F1-score (around 0.64). This result highlights the strong expressive power of Genetic Programming when trained on the full labeled dataset. The performance of GA is mainly driven by a high recall, indicating an effective detection of signal events, at the cost of a comparatively lower precision.

When Active Learning is introduced (GA+AL), a slight decrease in accuracy and F1-score is observed. This behavior can be explained by the reduced size of the labeled training set during the early generations, which limits the immediate expressive capacity of the model. Nevertheless, GA+AL maintains a balanced precision–recall trade-off, with recall remaining relatively high while precision stays close to that of the baseline GA.

Finally, the GA+AL+EL approach using soft voting does not yield additional performance improvements compared to GA+AL alone. Both approaches exhibit identical accuracy and F1-score values, suggesting that the diversity of the final Genetic Programming population is not sufficient for ensemble aggregation to consistently outperform the best individual model. Overall, these results emphasize a clear trade-off between maxi-

mum predictive performance achieved by GA and the significantly more computationally efficient GA+AL approach, while ensemble learning primarily contributes to result stabilization rather than systematic performance gains.FeNSE

A detailed analysis on the matter of Compar matter which will be done in the following of this section.

7.3 Advantages and Disadvantages of the Different Approaches

This section summarizes the strengths and weaknesses of each approach — baseline Genetic Algorithm (GA), Genetic Algorithm with Active Learning (GA+AL), and Genetic Algorithm with Active Learning and Ensemble Learning (GA+AL+EL) — based on the experimental results obtained.

Genetic Algorithm (GA)

The baseline GA achieves the highest predictive performance among all evaluated methods, with the best accuracy and F1-score. Its main strength lies in the expressive power of Genetic Programming when trained on the full labeled dataset, allowing the model to capture complex non-linear relationships in the data. In particular, GA exhibits a very high recall, making it effective at detecting signal events. However, this performance comes at a significant computational cost, as the training time is substantially higher than that of the other approaches. Moreover, the relatively lower precision indicates a higher number of false positives, which may be problematic in scenarios where misclassification costs are asymmetric.

Genetic Algorithm with Active Learning (GA+AL)

The GA+AL approach offers a favorable trade-off between performance and computational efficiency. While its accuracy and F1-score are slightly lower than those of the baseline GA, the degradation remains limited. A major advantage of GA+AL is the drastic reduction in training time, achieved by progressively increasing the labeled training set through uncertainty-based sampling. This makes GA+AL more scalable and suitable for large datasets or scenarios where labeling is expensive. On the downside, the reduced amount of labeled data during early generations can limit the expressive capacity of the model, leading to a moderate loss in predictive performance compared to the fully supervised GA.

Genetic Algorithm with Active Learning and Ensemble Learning (GA+AL+EL)

The GA+AL+EL approach aims to leverage the diversity of the final Genetic Programming population by aggregating multiple models through ensemble learning. Its main advantage is the potential stabilization of predictions by combining several individuals rather than relying on a single best model. However, in the present experimental setting,

ensemble learning does not lead to measurable improvements in accuracy, precision, recall, or F1-score compared to GA+AL alone. This suggests that the diversity within the final population is not sufficient to guarantee systematic performance gains. Consequently, while GA+AL+EL does not introduce additional computational overhead, its benefits remain limited in this context.

Overall, these results highlight a clear trade-off between maximum predictive performance (GA), computational efficiency with competitive performance (GA+AL), and robustness-oriented aggregation strategies (GA+AL+EL). The choice of approach therefore depends on the specific constraints of the application, such as available computational resources and labeling costs.

7.4 Detailed Analysis

Genetic Algorithm baseline (GA).

The GA baseline achieves an F1-score of approximately **0.644**, with a relatively high recall (approximately **0.789**), lower precision (approximately **0.544**) and relatively high accuracy (0.702). This behavior indicates a strong sensitivity to the positive class, at the expense of an increased number of false positives. The training time is significantly higher compared to the other approaches, as the full training dataset is used throughout all generations.

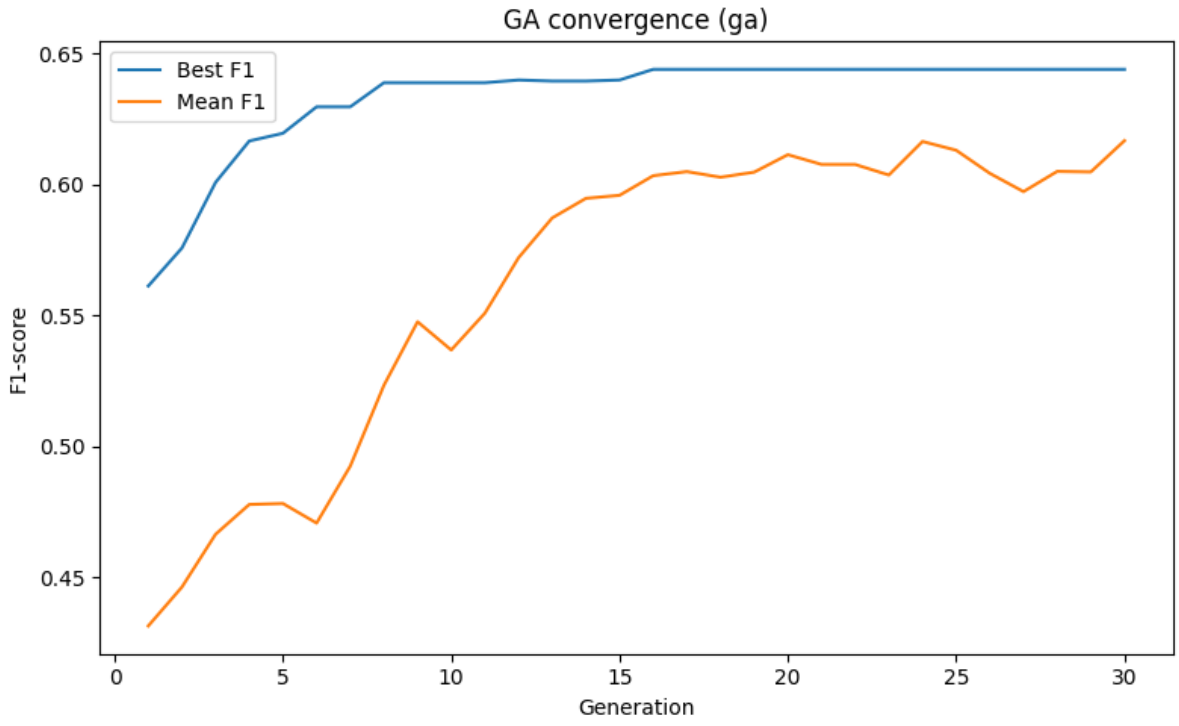

Figure 6: Evolution of F1 Metric throughout the GA training.

Genetic Algorithm with Active Learning (GA+AL).

The GA+AL approach reaches a comparable but however lower F1-score (approximately **0.629**), and the same assesment can be done for accuracy (0.695), recall (0.757), and precision (0.538). However, the training time has significantly reduces (more than 7 times faster). By starting from a smaller labeled subset and progressively expanding it through uncertainty-based sampling, this method achieves similar predictive performance with a substantially lower computational cost.

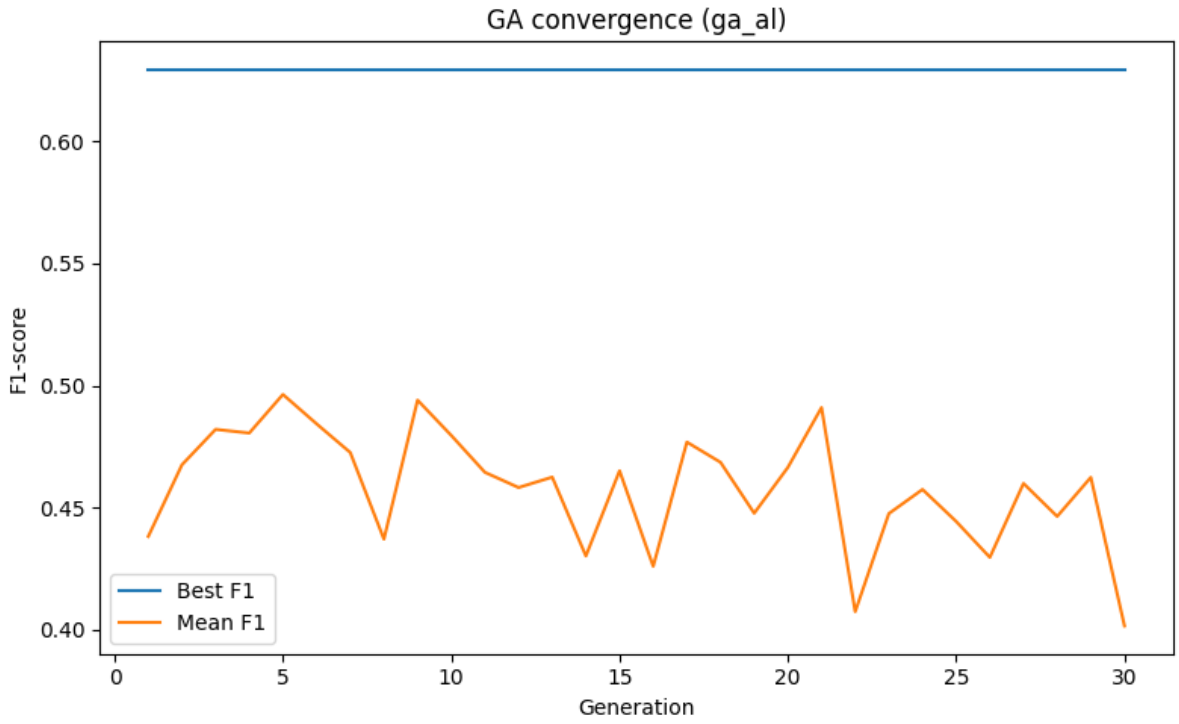

Figure 7: Evolution of F1 Metric throughout the GA+AM training.

Ensemble learning (GA+AL+EL).

The ensemble model based on GA+AL and soft voting does not yield a measurable performance improvement compared to the single GA+AL model. The metric results are basically the exact same for all four metrics. This result suggests that, in the retained configuration, the ensemble primarily stabilizes predictions rather than increasing predictive accuracy.

Overall, GA+AL is retained as a strong compromise between predictive performance and computational efficiency, while the GA baseline remains a useful reference model for comparison due to its slightly better performance.

7.5 Computational Cost and Efficiency

In addition to predictive performance, computational cost is a critical factor when evaluating machine learning pipelines, especially for iterative methods such as Genetic Programming. This section analyzes training time, data usage, and scalability across the different approaches.

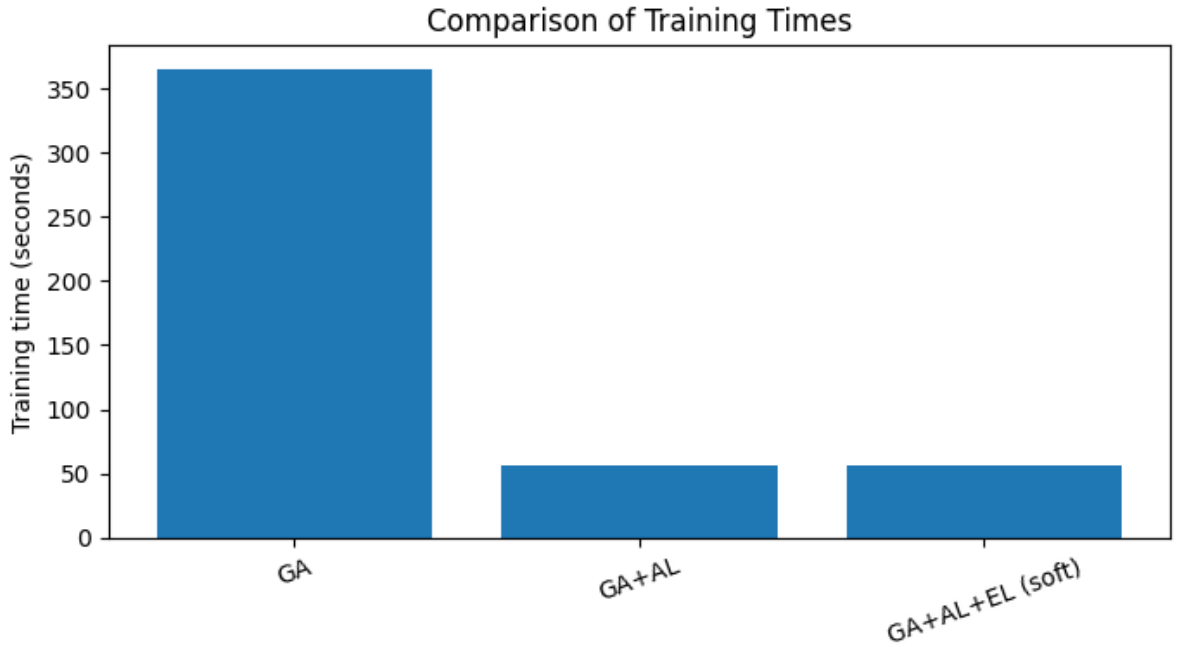

Figure 8: Comparison of the training times between baseline GA/GA+AL/GA+AL+EL models.

7.5.1 Baseline Genetic Algorithm

The baseline Genetic Algorithm is trained on the full labeled dataset, consisting of approximately 650 000 samples. As a result, it exhibits the highest computational cost among all approaches, with a total training time of approximately 400 seconds.

Despite this cost, the baseline GA serves as a strong reference point, providing a stable optimization process and consistent convergence behavior across generations.

7.5.2 Genetic Algorithm with Active Learning

The GA+AL approach dramatically reduces computational cost by limiting the initial labeled dataset to a fraction of the full training set. In this configuration, only 10% of the data is used initially, corresponding to approximately 65 000 samples.

This reduction leads to a training time of roughly 50 seconds, representing an more than 7-time faster training compared to the baseline GA. Importantly, this efficiency gain is achieved without a significant loss in predictive performance, demonstrating the effectiveness of active learning in prioritizing informative samples.

7.6 Impact of Experimental Variants

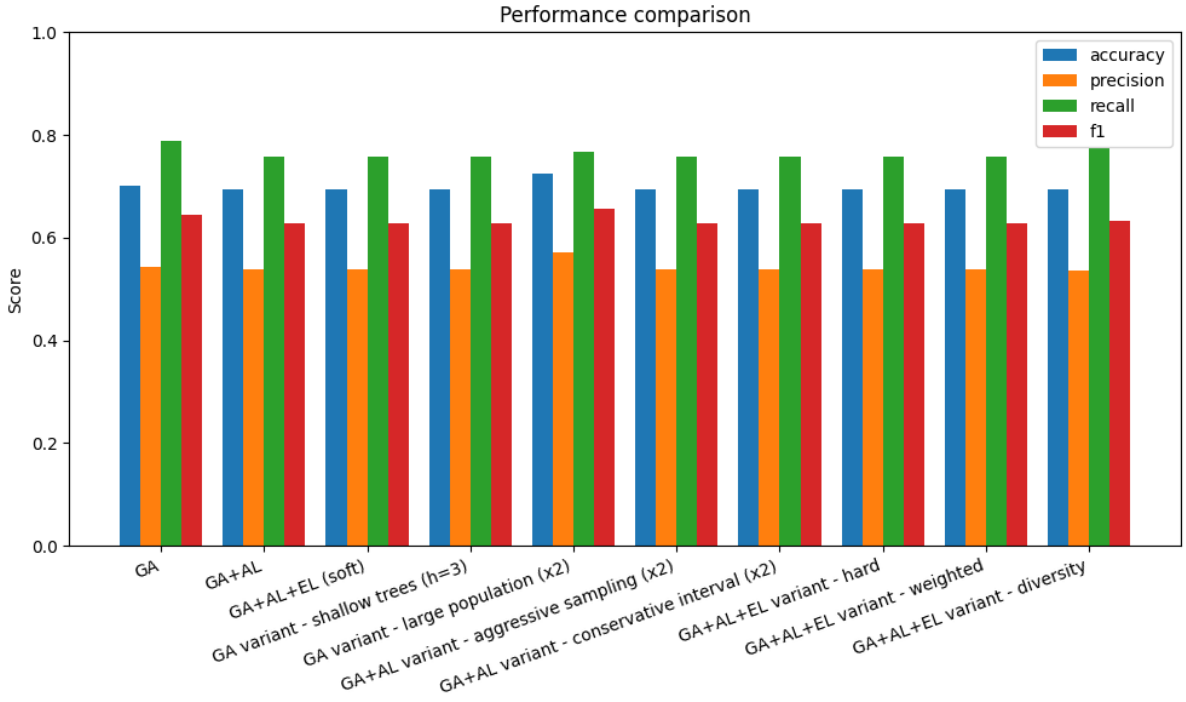

Figure 9: Performance comparison across experimental variants of GA, GA+AL and GA+AL+EL.

Experimental configurations further highlight the trade-offs between performance and cost:

- Increasing the population size (here by multiplying it by two) substantially increases training time, nearly doubling the computational cost while providing limited performance gains.
- Restricting tree depth (here by dividing it by two) improves generalization but does not significantly reduce training time, as the full dataset is still evaluated during evolution.
- Modifying active learning parameters has only a marginal impact on computational cost, confirming the robustness of the GA+AL baseline configuration.

The implementation of these variables are made in the `experiments.py` script, and its results are stored in the reports/results/experiments folder.

7.7 Ensemble Learning Overhead

Ensemble learning introduces minimal additional computational cost, as it operates exclusively at inference time. Since ensemble predictions reuse already trained individuals, the overhead is limited to multiple forward evaluations and aggregation steps.

Given this low cost, ensemble learning can be considered an efficient post-processing technique that improves prediction stability without affecting training complexity.

7.8 Summary

Overall, the results demonstrate that active learning is the primary driver of computational efficiency in the proposed pipeline. The retained production architecture achieves a favorable balance between training time and predictive performance, while experimental configurations provide valuable insights into potential future optimizations.

8 Limitations and Future Work

Despite the encouraging results obtained with the proposed pipeline, several limitations remain and open the door to meaningful future work.

8.1 Hyperparameter Optimization

One important limitation of the current approach lies in the manual selection of hyperparameters. Parameters such as population size, tree depth, mutation and crossover rates, as well as active learning settings, were chosen empirically and evaluated through a limited number of experimental variants.

Future work could involve a more systematic hyperparameter optimization strategy. In particular, defining structured dictionaries of model configurations and associated hyperparameters would allow automated and exhaustive experimentation. Such an approach would enable large-scale comparisons between different genetic programming configurations, leading to a more principled selection of optimal models.

8.2 Scalability and Computational Cost

Although active learning significantly reduces training time, genetic programming remains computationally expensive, especially when operating on large datasets. Evaluating entire populations across multiple generations imposes a substantial computational burden.

Several strategies could be explored to further mitigate this cost. These include parallelizing fitness evaluations, reducing population sizes dynamically during evolution, or introducing early stopping mechanisms based on convergence criteria. Additionally, surrogate models could be used to approximate fitness evaluations and limit the number of full evaluations required.

8.3 Search Space and Model Diversity

The expressiveness of genetic programming comes at the cost of a large and complex search space. While ensemble learning partially addresses this issue by leveraging population diversity, the exploration process itself remains challenging.

Future improvements could include adaptive operator probabilities, multi-objective optimization that jointly considers performance and model complexity, or the integration of domain-specific constraints to guide the evolutionary search more effectively.

8.4 Towards a More Automated Experimental Framework

Finally, the current experimental workflow relies on a predefined set of model variants. Extending this framework into a fully automated experimentation pipeline would allow continuous evaluation of new configurations as computational resources permit. Such a system could progressively refine both model performance and efficiency over time.

In summary, while the proposed architecture provides a solid and extensible baseline, future work could significantly enhance its robustness, scalability, and optimality through systematic hyperparameter exploration and improved computational efficiency.

9 Conclusion

In this project, we proposed and evaluated a complete machine learning pipeline based on genetic programming for a binary classification task on a large-scale dataset. The approach combined exploratory data analysis, robust preprocessing, evolutionary learning, active learning, and ensemble methods within a unified and reproducible framework.

The experimental results demonstrate that genetic programming is capable of learning meaningful decision functions despite the complexity of the data. Active learning significantly reduced training time while preserving comparable predictive performance, highlighting its relevance in large-scale settings. Ensemble learning, although not improving performance in this specific configuration, provided additional robustness and confirmed the stability of the selected models.

Beyond raw performance metrics, this work emphasized the importance of a structured experimental workflow. The clear separation between production models and experimental variants enabled systematic comparisons and informed model selection decisions, balancing predictive quality and computational cost.

Overall, the proposed architecture serves as a solid baseline for evolutionary learning on large datasets. While several avenues for improvement remain, particularly in hyperparameter optimization and computational efficiency, the current framework provides a strong foundation for further research and experimentation.